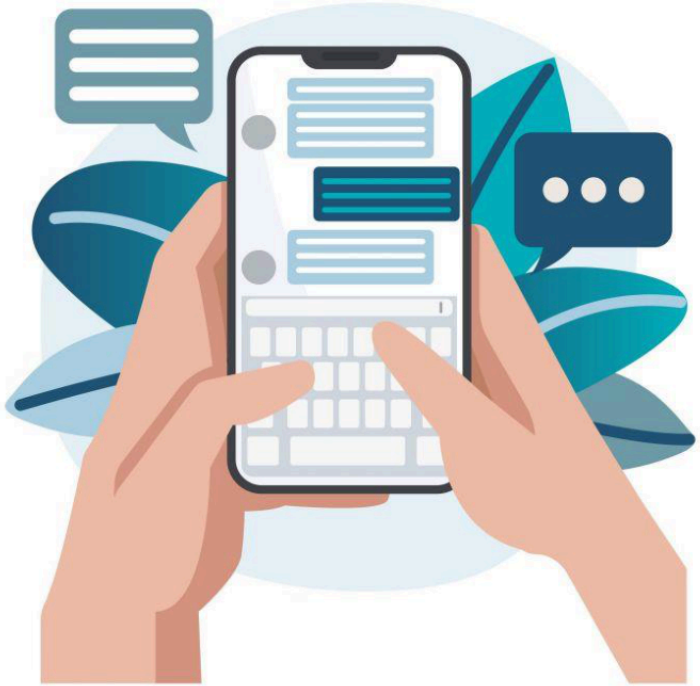
Client-Server based Instant Messenger



NEW YORK INSTITUTE
OF TECHNOLOGY
Vancouver

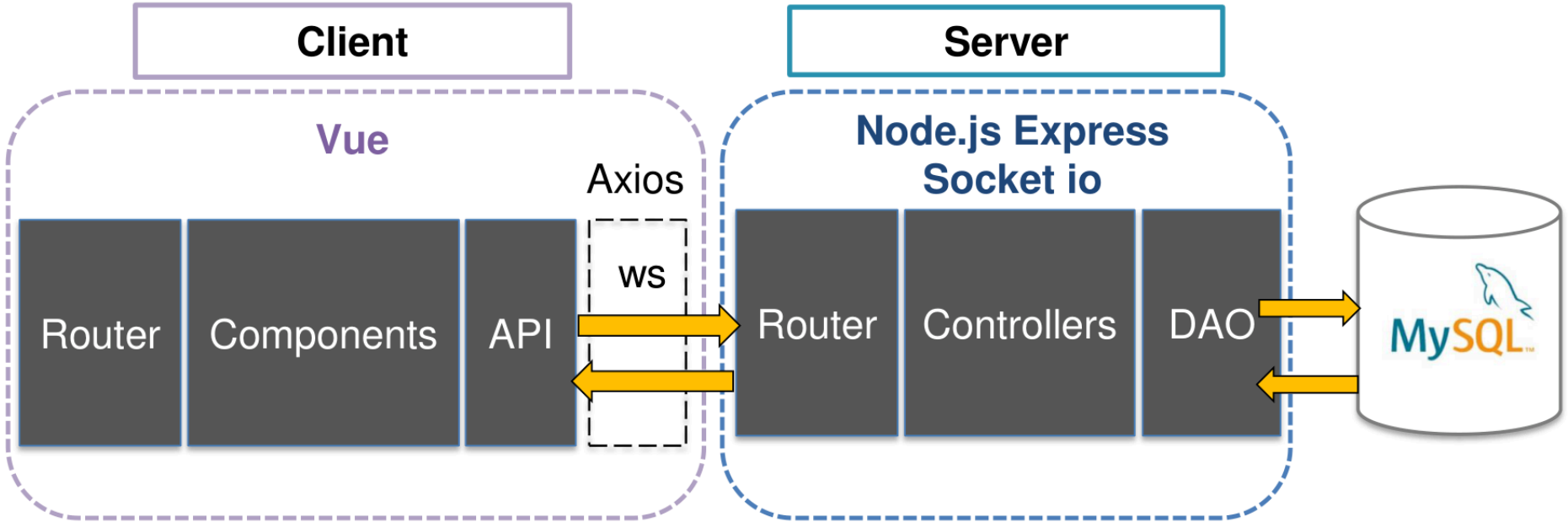
Yanqing Li (Lucis)

Client-Server based Instant Messenger

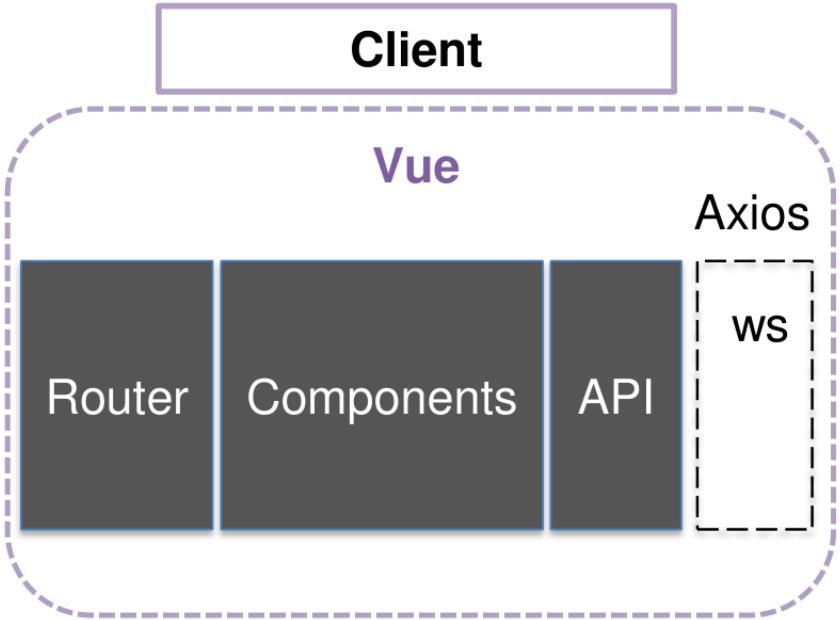
1. Introduction
2. What is WebSocket
3. WebSocket Vulnerabilities
4. WebSocket Defense



Project Introduction



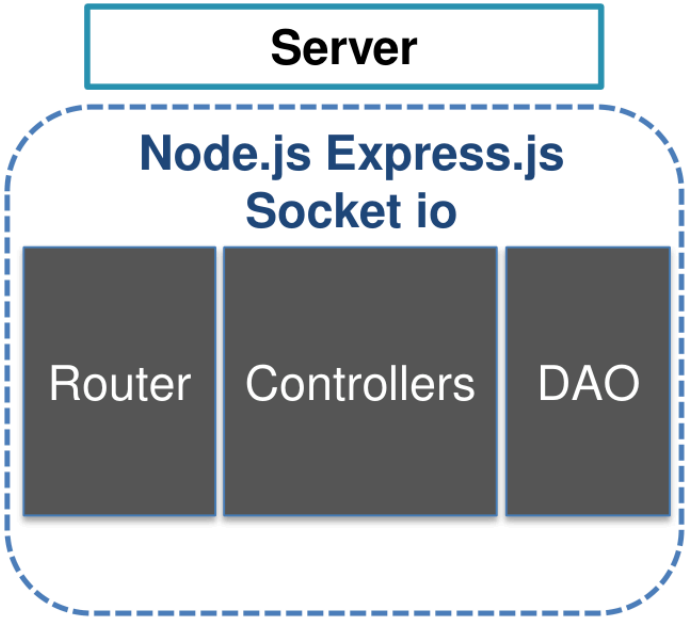
Project Introduction





Original chat screenshot removed
for participant privacy.
Client architecture retained.

Project Introduction



Node.js:

IS an open-source server environment; It's cross-platform.

Express.js:

A lightweight and flexible Node.js framework that provides a robust set of features for building web applications.

Socket.IO:

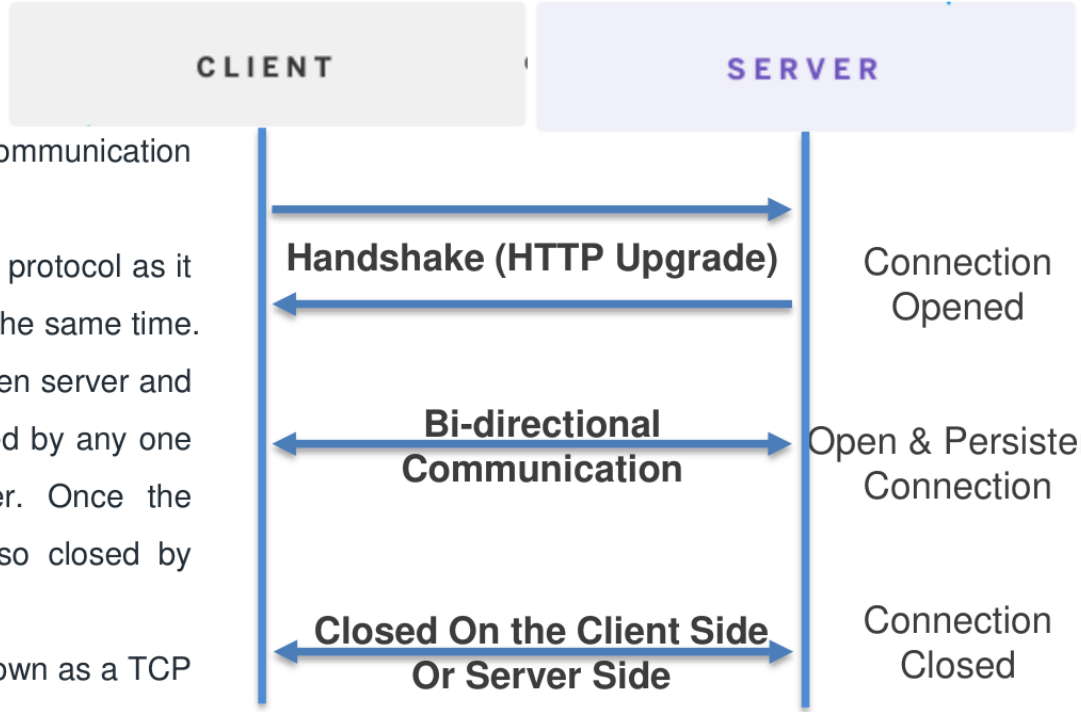
Enables real-time bidirectional event-based communication. Socket abstract WebSocket connections. It uses WebSocket for transportation and automatically falls back to long polling if WebSocket isn't supported by the client.

WebSocket

What is WebSocket - RFC6455

Is a communication protocol that is mainly used for communication between clients and a server.

- **Full-Duplex Protocol:** WebSocket is a full-duplex protocol as it allows the application to send and receive data at the same time.
- **Stateful Protocol:** It means the connection between server and client will not be terminated until and unless closed by any one of them either by the client or by the server. Once the connection is terminated from one end, it is also closed by another end.
- **3-way handshake:** It uses a 3-way handshake known as a TCP connection.



WebSocket Connection between Client & Server

WebSocket

WebSocket Applications

WebSocket is suitable whenever fast online connections are required. Nowadays, many sites rely on real-time connections to offer certain services. These include:

- **Online Education Sector**
- **Online Gaming: Collaborative applications**
- **Real-time data visualization**
- **Event Update applications**
- **Tracking User Behaviour**



Mozilla has launched a real-time Massively multiplayer HTML5 based Game called “BrowserQuest” that uses Websockets and Canvas.

WebSocket

Why WebSocket?

- WebSocket is a **naturally** full-duplex, bidirectional, single-socket connection. With WebSocket, your HTTP request becomes a single request to open a WebSocket connection and reuses the same connection from both sides;

- WebSocket makes real-time communication much **more efficient**. Instead of using polling over HTTP to receive notifications, WebSocket saves bandwidth and CPU power.



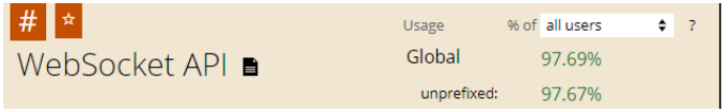
HTTP	WebSocket
Duplex	
Half	Full
Messaging Pattern	
Request-reponse	Bi-directional
Service Push	
Not natively supported. Client polling or streaming download techniques used.	Core feature
Overhead	
Moderate overhead per request/connection.	Moderate overhead to establish & maintain the connection, then minimal overhead per message.
Intermediary/Edge Caching	
Core feature	Not possible
Supported Clients	
Broad support	Modern languages & clients

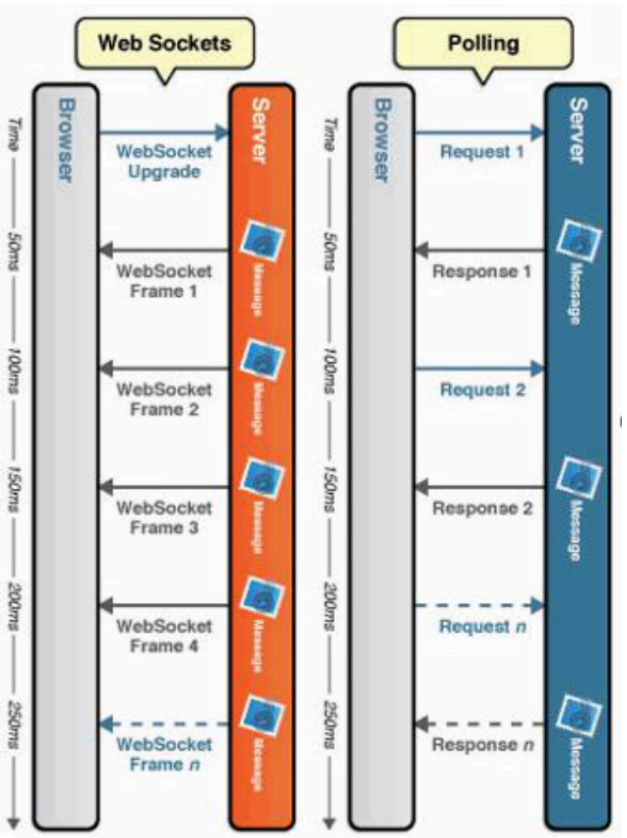
WebSocket

Why WebSocket?

- WebSocket **reduces latency**. For example, unlike polling, WebSocket makes a single request. The server does not need to wait for a request from the client. Similarly, the client can send messages to the server at any time. This single request greatly reduces latency over polling, which sends a request at intervals, regardless of whether messages are available.

- WebSocket is about Simplicity and Most of the **browsers support** WebSocket (97.69% of all browsers in 2022)

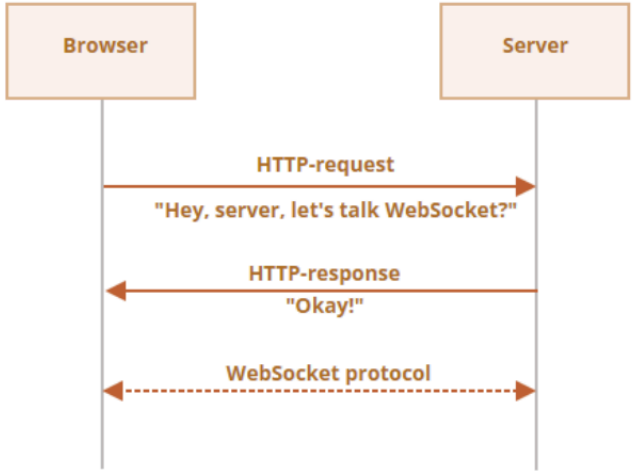


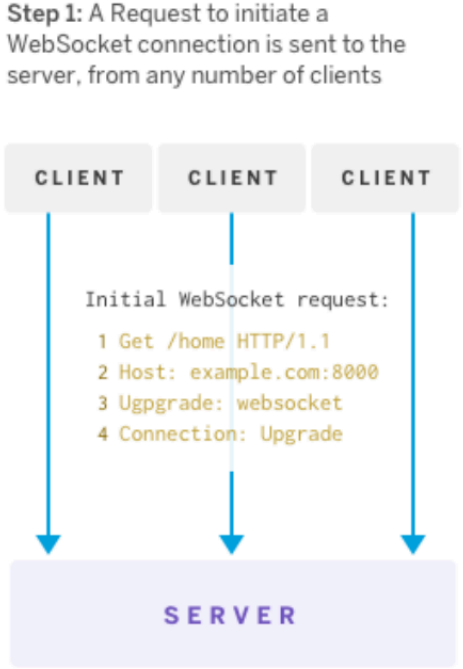


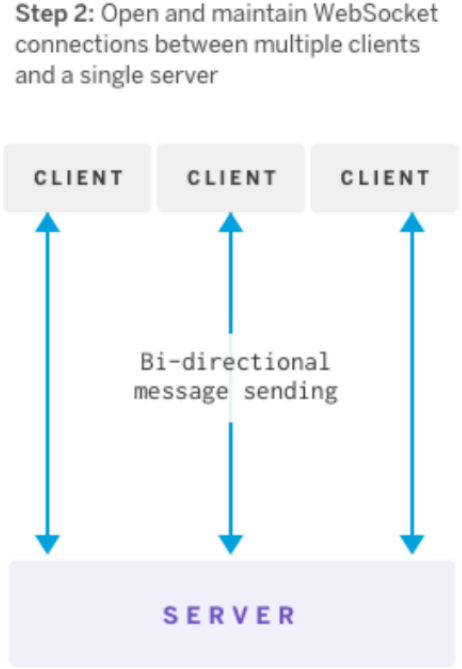
NEW YORK INSTITUTE OF TECHNOLOGY
Vancouver

WebSocket

How ? To establish a WebSocket connection, the client and server upgrade from the HTTP protocol to the WebSocket protocol during their initial handshake, as shown in the following:







WebSocket

Here's an example of browser headers for a request made by new WebSocket

The client sends the request:

```
GET /chatService HTTP/1.1
Host: server.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

- Origin: allows the server to decide whether or not to talk WebSocket with this website.
- Upgrade: the requested protocol is "WebSocket"
- Connection: signals that the client would like to change the protocol.
- Sec-WebSocket-key: a random browser-generated key used to ensure that the server supports WebSocket protocol.
- Sec-WebSocket-Version: WebSocket protocol version

If the server agrees to switch to WebSocket, it should send code 101 response

The server answers:

```
HTTP/1.1 101 Switching Protocols
Upgrade: WebSocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBi
Sec-WebSocket-Protocol: superc
```

Here `Sec-WebSocket-Accept` is `Sec-WebSocket-Key`, recorded using a special algorithm. Upon seeing it, the browser understands that the server really does support the WebSocket protocol. Afterwards, the data is transferred using the WebSocket protocol,

The new URL scheme for websites that use WebSocket is *ws* instead of *http*. There is also an equivalent for secure connections: *wss* instead of *https*.

WebSocket

WS Events:

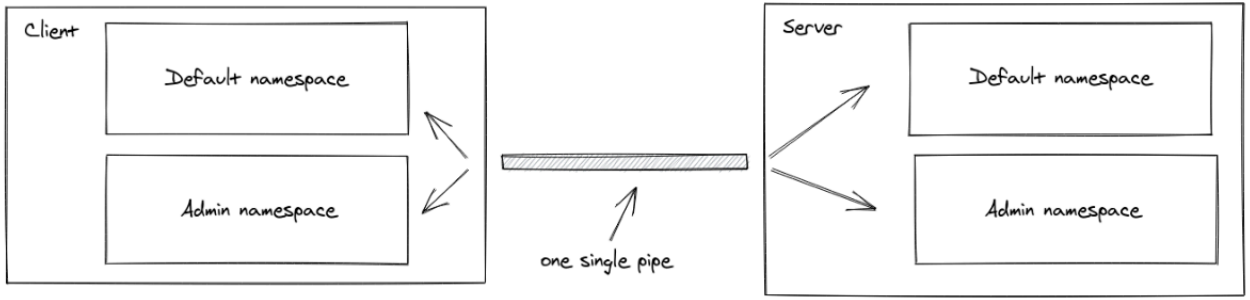
Once the socket is created, we should listen to events on it. There are totally 4 events:

1. open – connection established 2. message – data received; 3.error – WebSocket error; 4.Close – connection closed.

Here are the features provided by Socket.IO over plain WebSockets:



- reliability (fallback to HTTP long-polling in case the WebSocket connection cannot be established)
- automatic reconnection
- packet buffering: By default, any event emitted while the Socket is not connected will be buffered until reconnection.
- Acknowledgments: Events are great, but in some cases, you may want a more classic request-response API. In Socket.IO, this feature is named acknowledgements.
- broadcasting to all clients or to a subset of clients (what we call "Room")
- multiplexing (what we call "Namespace"): A Namespace is a communication channel that allows you to split the logic of your application over a single shared connection.



WebSocket Vulnerabilities

- **WebSocket Authorization/Authentication Attacks:** WebSockets don't manage authorization and authentication. Application-level protocols should handle this separately, especially when sensitive data is being transferred. The WebSocket protocol also doesn't let servers authenticate the client during the handshake process. So, threat actors hunt for mismanaged authorization and authentication in order to circumvent a business's security controls and start stealing data.
- **Sniffing and Skimming Attacks:** WebSockets trade data in plain text, in the same way as HTTP does. This makes the data extremely easy for threat actors to steal using packet sniffers. Packet sniffers are essentially a specific form of e-skimming attack. Detecting WebSocket vulnerabilities in web applications and pages is a tedious manual task. Application developers and cybersecurity professionals need to ensure that they are using the WebSocket Secure (wss://) protocol to ensure that data is being encrypted using transport layer security (TLS).

WebSocket Vulnerabilities

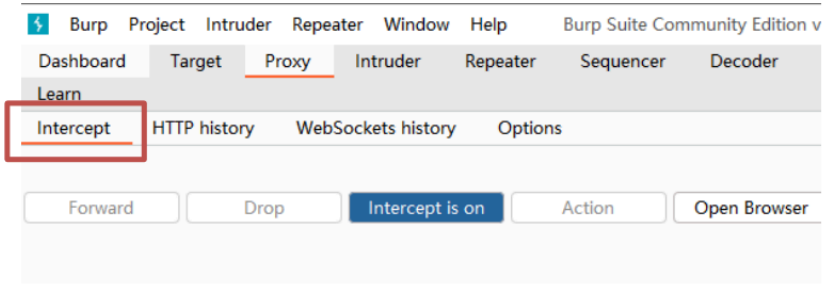
- **Tunneling and Cross-site Scripting Attacks:** Anybody can use WebSockets to tunnel into a TCP service. Cross-site scripting (XSS) is a popular client-side attack that frequently evolves into a tunneling attack. Threat actors love to deploy cross-site scripting attacks, during which they inject malicious scripts into websites. By sending malicious scripts from web applications to different users, threat actors can create database connections directly through the browser. The net outcome is that they have direct access to data and can start collecting it and using it at will.
- **Data Masking:** Depending on how a business uses Websockets and how they are deployed, WebSockets can allow an unlimited number of connections to run between the client-side and the server-side. If a threat actor manages to get a hold over WebSockets, they can easily run a DoS attack, flooding servers to exhaust computing resources and slow down web applications and pages. The net outcome is that your customers either can't do business with you, or can only do business with you slowly. Website and web application speed is extremely important. If doing business with you becomes a slow chore, customers will jump very quickly to your competitors.

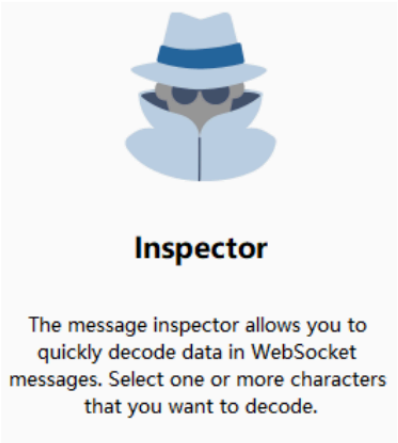
WebSocket Vulnerabilities

- **Attack Demo:**

- Burp or Burp Suite is a set of tools used for penetration testing of web applications. It is the most popular tool among professional web app security researchers and bug bounty hunters.

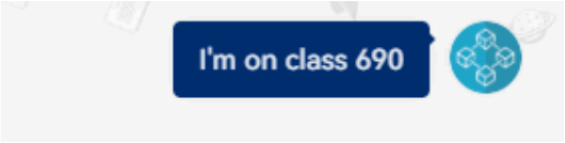
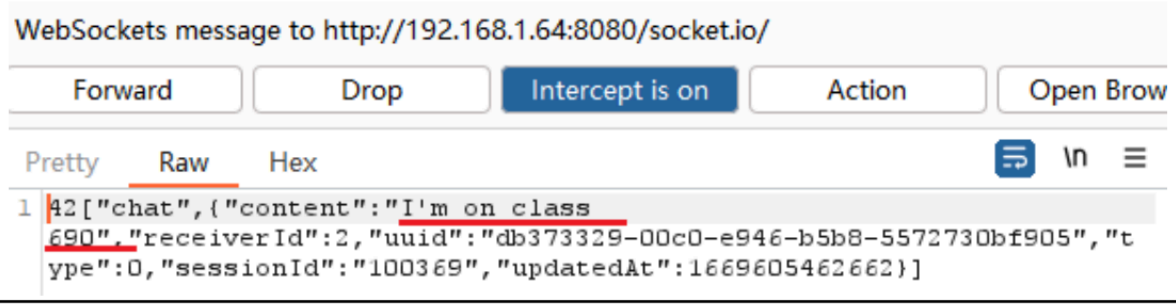
- BurpSuite contains an intercepting proxy that lets the user see and modify the contents of requests and responses while they are in transit. It also lets the user send the request/response under monitoring to another relevant tool in BurpSuite, removing the burden of copy-paste.





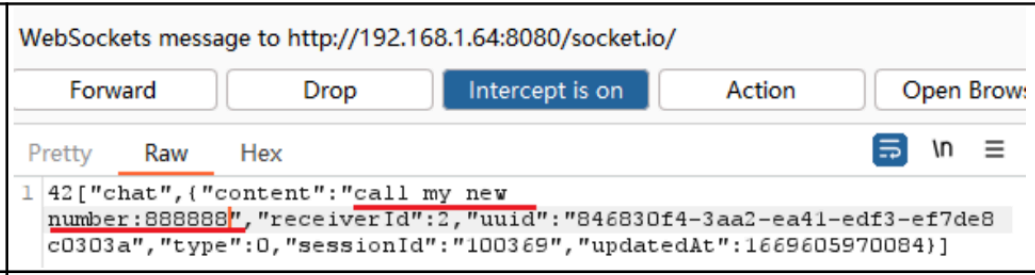
WebSocket Vulnerabilities

- **Attack Demo:**

User A sends a message to User B.	
BurpSuite intercepts the transaction and finds the message from User A.	

WebSocket Vulnerabilities

- **Attack Demo:**

Changed the original message.	
User B received the faked message.	

How to improve WebSocket Security

WebSocket security measures might be a bit of a misnomer. They rather should be categorized as a specific type of application security or cybersecurity hygiene. Here are the main ones to implement and continuously nurture:

- Always use WebSockets over TLS (wss://).
- Make sure to check the origin header during the handshake process and use the Access-Control-Allow-Origin header on the server side.
- Protect the handshake against CSRF attacks using Nonce/CSRF tokens.
- Consistently deploy and enforce authentication best-practices for WebSockets.
- Make sure to deploy data model input validation of messages in both directions from the client to server, and from server to client.
- Always encode the output of messages when embedded in the web application.

Reference

- https://www.geeksforgeeks.org/what-is-the-use-of-websocket-api/
- https://javascript.info/websocket
- https://www.ionos.com/digitalguide/websites/web-development/what-is-websocket/
- https://brightsec.com/blog/websocket-security-top-vulnerabilities/
- https://www.geeksforgeeks.org/what-is-burp-suite/